Allan Batista
Victor Nascimento
Rodrigo Peres

CLASSIFICAÇÃO DE PRODUTOS NO E-COMMERCE UTILIZANDO DADOS HETEROGÊNEOS

Rio de Janeiro

2020, v-1.0.0

Allan Batista
Victor Nascimento
Rodrigo Peres

CLASSIFICAÇÃO DE PRODUTOS NO E-COMMERCE UTILIZANDO DADOS HETEROGÊNEOS

Trabalho de pesquisa e desenvolvimento na B2W Digital para a classificação de Produtos no e-commerce.

B2W Digital

Pesquisa e desenvolvimento

Rio de Janeiro

2020, v-1.0.0

Resumo

A classificação de produtos no **e-commerce** é um desafio devido a sua variabilidade e qualidade das informações disponíveis. Este trabalho propõem uma arquitetura híbrida de **Rede Neural Artificial Profunda** para **dados heterogêneos e não estruturados** (texto e imagem) em uma única arquitetura que combina **Redes Neurais Convolucionais** e **Arquitetura "Transformer"** com uma estrutura de **classificação multinível**.

Palavras-chave: aprendizado de máquina, redes neurais profundas, redes convolucionais, arquitetura transformer.

Lista de ilustrações

Figura 1 – Quantidade de Produtos ao longo dos Meses 2018+ 7
Figura 2 – Produtos similares em categorias distintas 8
Figura 3 – Estrutura das categorias em árvore 9
Figura 4 – Quantidade de Produtos Por Departamento 10
Figura 5 – Arquitetura Macro do Modelo 12
Figura 6 – Arquitetura Macro do Modelo de texto 13
Figura 7 – Representação das palavras pelos seus n-grams 14
Figura 8 – SKIP-GRAM Model 15
Figura 9 – Visualização da Distorção de Texto 16
Figura 10 – Arquitetura Transformer 17
Figura 11 – Bloco Codificador da Arquitetura Transformer 17
Figura 12 – Exemplo de utilização do mecanismo de atenção em uma sequência . . 19
Figura 13 – Scaled Dot-Product Attention e Muilt-Head Attention 20
Figura 14 – Arquitetura do modelo de Imagem 21
Figura 15 – Distorção de Imagens 21
Figura 16 – Bloco Residual 22
Figura 17 – Arquiteturas dos blocos convolucionais da ResNet 23

Lista de tabelas

Tabela 1 – Quantidade de Categorias 7
Tabela 2 – Amostra de categorias ambíguas no contexto de Mouse para computador 9
Tabela 3 – Resultados dos Experimentos 25
Tabela 4 – Configurações de Treinamento do FastText 32

Sumário

1 INTRODUÇÃO . . . 7
1.1 PORQUE UTILIZAR TEXTO E IMAGEM . . . 8
1.2 ESTRUTURA DAS CATEGORIAS . . . 8
1.3 PROBLEMAS NOS DADOS . . . 9
1.3.1 AMBIGUIDADES . . . 9
1.3.2 RUÍDOS NOS DADOS . . . 9
1.3.3 DESBALANCEAMENTO NAS CATEGORIAS . . . 10
1.3.4 LÍNGUA PRÓPRIA . . . 10

2 IMPLEMENTAÇÃO . . . 12
2.1 MODELO DE TEXTO . . . 12
2.1.1 LIMPEZA E NORMALIZAÇÃO . . . 13
2.1.2 VETORIZAÇÃO . . . 13
2.1.2.1 REPRESENTAÇÃO DAS PALAVRAS . . . 13
2.1.2.2 MODELO FASTTEXT . . . 14
2.1.3 DISTORÇÃO DO TEXTO . . . 15
2.1.4 ARQUITETURA TRANSFORMER . . . 16
2.1.4.1 EMBEDDING INPUT . . . 18
2.1.4.2 CODIFICAÇÃO POSICIONAL . . . 18
2.1.4.3 ATTENTION MECHANISM . . . 19
2.1.4.4 MULTI-HEAD ATTENTION . . . 20
2.2 IMAGEM . . . 20
2.2.1 DISTORÇÃO DAS IMAGENS . . . 21
2.2.2 ARQUITETURA RESNET . . . 22
2.3 CONJUNTO DE DADOS . . . 23
2.3.1 SELEÇÃO DO CONJUNTO DE DADOS . . . 23
2.4 OTIMIZAÇÃO E FUNÇÃO DE ERRO . . . 24
2.4.1 OTIMIZADOR . . . 24
2.4.2 FUNÇÃO DE ERRO (LOSS FUNCTION) . . . 24

3 RESULTADOS . . . 25
3.1 MÉTODOS DE AVALIAÇÃO . . . 25
3.2 CONFIGURAÇÃO DO TREINAMENTO . . . 25
3.3 EXPERIMENTOS . . . 25

4 CONCLUSÃO . . . 26

REFERÊNCIAS 27

APÊNDICES 28

APÊNDICE A – FUNÇÃO DE NORMALIZAÇÃO DE TEXTO . . 29

APÊNDICE B – FUNÇÃO DE DISTORÇÃO DE TEXTO 30

APÊNDICE C – FUNÇÃO DE DISTORÇÃO DE IMAGEM 31

APÊNDICE D – FASTTEXT MODELO PRÉ-TREINADO 32
D.1 Configurações do Treinamento 32
D.2 Modelos Pré-Treinados 32
D.3 Corpus 32

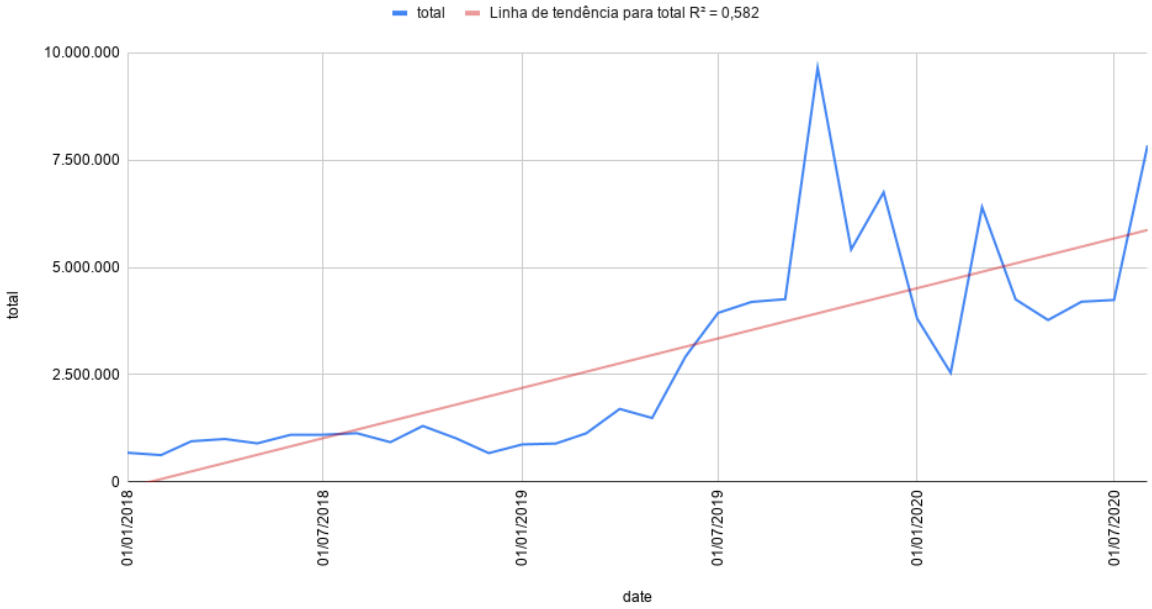
APÊNDICE E – CODIFICAÇÃO POSICIONAL 34

APÊNDICE F – FOCAL LOSS 35

1 Introdução

Com continuo crescimento do e-commerce no Brasil nos últimos anos e a adesão cada vez maior de novas lojas no Marketplace, traz consigo um aumento considerável na entrada de novos produtos no site da companhia. Tal confirmação pode ser observado na Figura 1.

Figura 1 – Quantidade de Produtos ao longo dos Meses 2018+



Fonte: Elaborado pelo Autor

Quando os produtos entram no marketplace, eles precisam ser categorizados à fim de proporcionar uma melhor qualidade do catálogo ao cliente final ao buscar um produto no site. No entanto, existe uma quantidade expressiva de categorias, que pode ser observado na Tabela 1, o que gera muita confusão e erros das lojas ao categorizar um produto na estruturas das categorias.

Tabela 1 – Quantidade de Categorias

Departamento	60
Setor	1152
Família	4595
Subfamília	8885

Fonte: Elaborado pelo Autor

Portanto, se faz necessário a criação de um sistema capaz de fazer esta categorização de forma automática a fim de reduzir o custo, tempo e elevar a assertividade da classificação de produtos na entrada de novos itens no marketplace.

Para a resolução deste problema, foi desenvolvido uma arquitetura de Redes Neurais Artificiais utilizando dados heterogêneos não estruturados (texto e imagem) dos produtos a fim de tornar a classificação mais eficiente do ponto de vista de assertividade.

1.1 PORQUE UTILIZAR TEXTO E IMAGEM

A necessidade de utilizar dois tipos de dados tão distintos durante a classificação, se faz necessário devido em alguns casos esses dados individualmente não possuírem informação o suficiente para classificar adequadamente a categoria do produto. Pode ser visto um exemplo de ambiguidade de produtos na Figura 2 em categorias distintas.

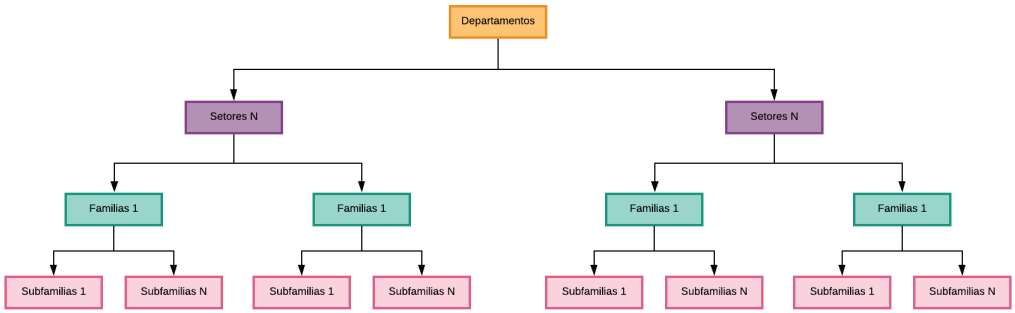
Figura 2 – Produtos similares em categorias distintas

Fonte: Elaborado pelo Autor

1.2 ESTRUTURA DAS CATEGORIAS

A estrutura das categorias é em formato de árvore e pode ser visto na Figura 3, onde os níveis superiores são mais genéricos enquanto os níveis mais baixos são mais específicos. Hoje existe um conjunto total de 14692 categorias.

Figura 3 – Estrutura das categorias em árvore



Fonte: Elaborado pelo Autor

1.3 PROBLEMAS NOS DADOS

Como todo conjunto de dados do mundo real, o nosso conjunto de dados não é diferente e possui diversos problemas. Ambiguidades de Categorias, Ruídos e desbalanceamento. A seguir, será feito um resumo dos principais problemas e dificuldades encontrados.

1.3.1 AMBIGUIDADES

A ambiguidade de categorias é outro problema encontrado no nosso conjunto dados, isso de deve a criação desordenada e descentralizada de categorias ao longo dos anos para atender diversos setores da companhia com objetivos distintos.

Tabela 2 – Amostra de categorias ambíguas no contexto de Mouse para computador

DEPARTAMENTO	**SETOR**	**FAMÍLIA**	**SUBFAMÍLIA**
INFORMÁTICA ACESSÓRIOS	MOUSE	MOUSE COM FIO	MOUSE COM FIO
INFORMÁTICA ACESSÓRIOS	MOUSE	MOUSE WIRELESS	MOUSE WIRELESS
INFORMÁTICA ACESSÓRIOS	MOUSE	MOUSE RETRÁTIL	MOUSE RETRÁTIL
PC GAMER	MOUSE GAMER	MOUSE COM FIO	MOUSE COM FIO
PC GAMER	MOUSE GAMER	MOUSE WIRELESS	MOUSE WIRELESS
PC GAMER	COMBO GAMER	TECLADO E MOUSE	TECLADO E MOUSE
PC GAMER	COMBO GAMER	MOUSE + MOUSEPAD GAMER	MOUSE + MOUSEPAD GAMER

Fonte: Elaborado pelo autor

1.3.2 RUÍDOS NOS DADOS

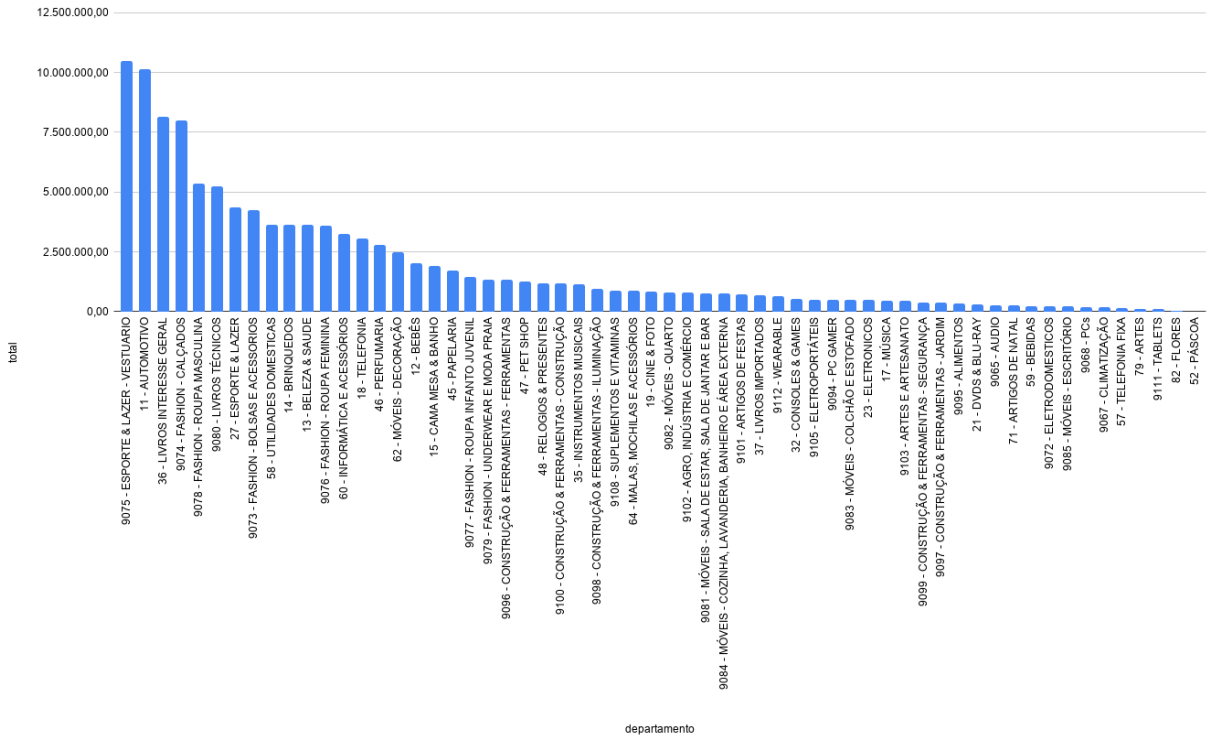
Além dos problemas de ambiguidades, há problemas também de produtos mal categorizados anteriormente, onde por algum equívoco (dificuldade/ambiguidade), um produto foi alocado em uma categoria erroneamente.

Podemos citar um exemplo de que um celular, que poderia estar alocado em televisões. Se este item for selecionado para o treinamento, logo fará com que outros celulares sejam classificados como televisão, um outro problema é uma distorção durante a avaliação dos produtos, pois se esse produto foi selecionado para o conjunto de teste do modelo, logo ele seria classificado como celular, mas a categoria atrelada à ele seria a de televisão.

1.3.3 DESBALANCEAMENTO NAS CATEGORIAS

O problema de desbalanceamento de dados também se encontra presente, para mitigar esse problema utilizamos algumas técnicas como: oversampling, undersampling e uma função de erro especializada para dados desbalanceados.

Figura 4 – Quantidade de Produtos Por Departamento



Fonte: Elaborado pelo Autor

1.3.4 LÍNGUA PRÓPRIA

Os títulos dos produtos são tão distintos nas palavras e na forma de escrever da linguá comum, que partimos do princípio que estamos trabalhando como uma língua distinta de todas as outras. Pois não conseguimos aplicar nenhuma técnica tradicional de Processamento de Linguagem Natural como: lemmantize, stemmer ou mesmo stop-words.

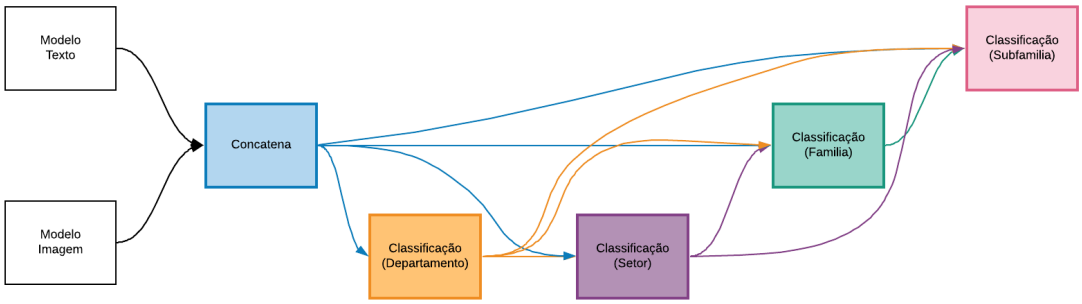
- Smartphone Samsung Galaxy S10e 128GB Dual Chip Android 9.0 Tela 5,8"Octa-Core 4G Câmera 12MP + 16MP - Preto
- Geladeira/Refrigerador Electrolux DC35A Branca 260L Cycle Defrost - 220v
- Tapete Para Sala Shaggy Requinte Casa Dona 100x150cm Cinza

É possível observar nos exemplos acima que não seguem nenhuma regra gramatical do português. Além de quê, com a entrada de novos produtos de outros países, presumir regras gramaticais parece imprudente.

2 IMPLEMENTAÇÃO

O modelo proposto é uma arquitetura de **Rede Neural Artificial Profunda** com dados heterogêneos não estruturados. Seu formato é a combinação de outras arquiteturas já conhecidas e amplamente utilizadas para resolver separadamente problemas de **Processamento de Linguagem Natural** e **Visão Computacional**.

Figura 5 – Arquitetura Macro do Modelo



Fonte: Elaborado pelo Autor

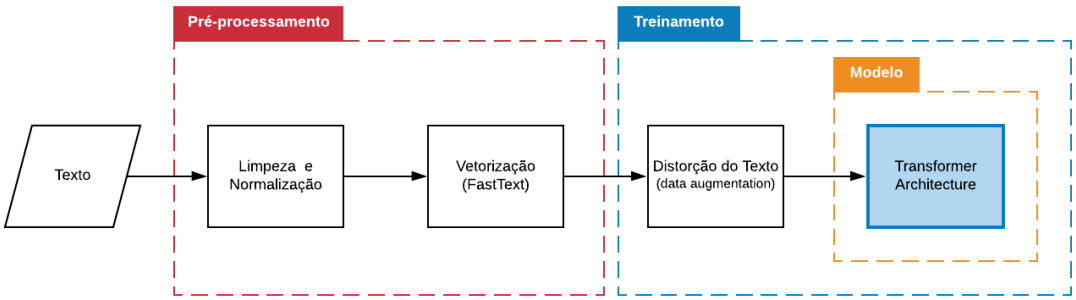
Para viabilizar a utilização de dados heterogêneos simultaneamente no modelo, são utilizados dois blocos distintos na rede neural para a extração dos atributos. Para então combinar esses atributos em um conjunto maior que serão injetados nas redes densas de classificação. Vale notar que a saída do nível acima faz parte da entrada do próximo nível.

2.1 MODELO DE TEXTO

O modelo responsável por extrair os atributos dos textos dos produtos, utiliza várias técnicas para atingir este objetivo e pode ser visto na Figura 6.

O texto antes do treinamento por **limpeza e normalização**, **vetorização** e durante o treinamento o texto recebe **distorções** em toda interação e então passa pela arquitetura **Transformer**.

Figura 6 – Arquitetura Macro do Modelo de texto



Fonte: Elaborado pelo Autor

2.1.1 LIMPEZA E NORMALIZAÇÃO

Todo o texto utilizado passa por uma função de limpeza e normalização a fim de remover a maior quantidade de inconsistências e simplificar seu formato. A função de normalização pode ser encontrada no Apêndice A.

2.1.2 VETORIZAÇÃO

A representação do texto de forma vetorial para que possa ser utilizada nos modelos, precisam ser simples e que contenham em si as informações sobre o significado das palavras.

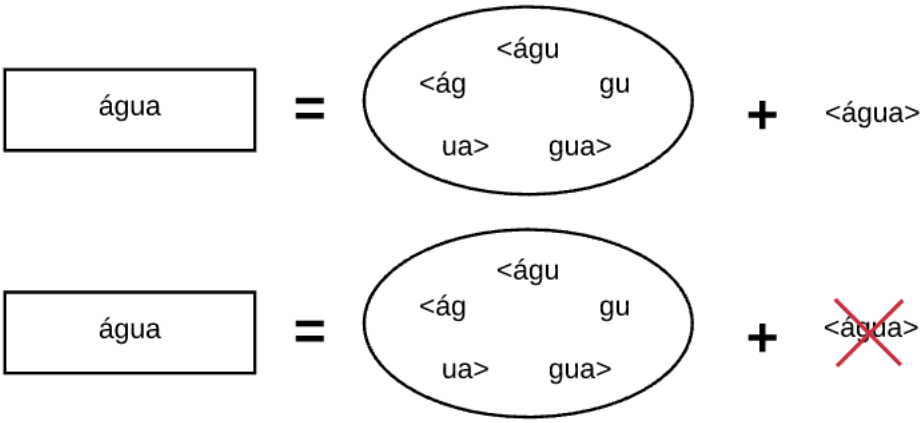
Para o nosso caso, obtivemos melhores resultados usando o **FasText** proposto no artigo (BOJANOWSKI et al., 2016). Esse método de representação vetorial é interessante, pois diferente do **CBOW** e **SKIP-GRAM** introduzidos no artigo (MIKOLOV et al., 2013), o FastText é capaz de representar palavras que não foram vistas durante o treinamento o que é uma grande vantagem no nosso contexto pois muitos termos seguem um padrão de formação, como no exemplo à seguir, onde apenas muda um carácter no modelo do smartphone.

- Smartphone Samsung Galaxy **S10** ...
- Smartphone Samsung Galaxy **S10e** ...
- Smartphone Samsung Galaxy **S10+** ...

2.1.2.1 REPRESENTAÇÃO DAS PALAVRAS

As palavras são representadas pela soma da representação vetorial de seus *n-grams* mais um termo especial que é a própria palavra se ela estiver presente no dicionário. Caso

Figura 7 – Representação das palavras pelos seus n-grams



Fonte: Elaborado pelo Autor

a palavra não esteja presente no dicionário, então ela só será representada somente pela soma de seus *n-grams*.

Note que na primeira parte da Figura 7 a palavra "água" aparecer como um termo especial, enquanto na segunda parte o termo especial não aparece, dando a ideia de que não esta presente no dicionário, portanto a representação da palavra será feita somente pelos seus *n-grams*.

Vale notar que o FastText já possui diversos modelos pré-treinados em várias línguas e podem ser encontradas no seu site FastText Models.

No entanto, para maximizar a performance do modelo, foi necessário fazer um treinamento especializado com os dados do e-commerce. Para mais detalhes, veja no Apêndice D.

2.1.2.2 MODELO FASTTEXT

O modelo do FastText é uma variação do **skip-gram** introduzido no (MIKOLOV et al., 2013)que tem como objetivo prever as palavras ao seu redor de forma a maximizar as suas probabilidades. Um exemplo dessa ideia pode ser visto na Figura 8.

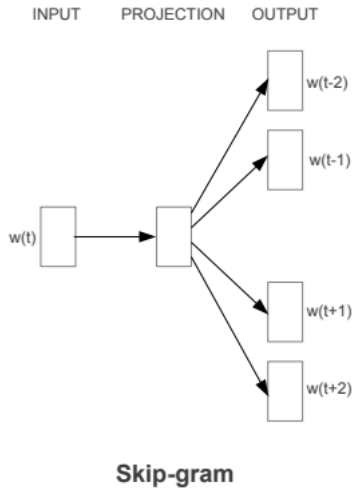
A definição do modelo do skip-gram é dado pela seguinte função:

$$\Sigma_{t=1}^{T} = \left[\Sigma_{c \in C_t} l(s(w_t, w_c))) + \Sigma_{n \in N_{t,c}} l(-s(w_t, n))\right]$$

Tal que w_t são as palavras em volta e w_c é o contexto e n são amostras randômicas negativas e l é a **função logística de custo** (logistic loss function).

$$l(x) = log(1 + e^{-x})$$

Figura 8 – SKIP-GRAM Model



Fonte: Disponível em (MIKOLOV et al., 2013)

O modelo de "subword"(sub-palavra) é dado pela função:

$$s(w, c) = \sum_{g \in G_w} z_g^T v_c$$

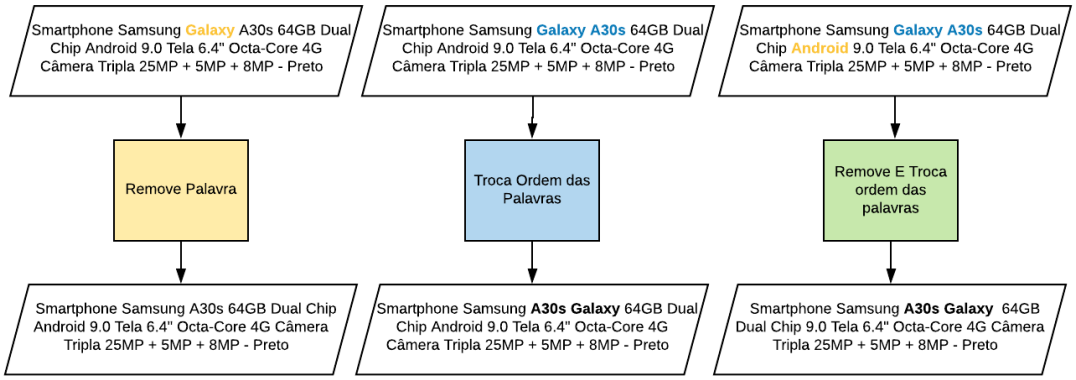
Onde G é o tamanho do dicionário de *n-grams* e dado um w (palavra) as suas partes estarão contidas em G, $G_w \subset 1, ..., G$. A representação vetorial de cada n-gram é dado por z_g e a representação da palavra em si, será o seu somatório.

2.1.3 DISTORÇÃO DO TEXTO

Para tornar o treinamento mais genérico possível e com maior variabilidade de dados, são utilizados dois tipos de distorção no texto (data augmentation), sendo a primeira de remover palavras e a segunda inverter a ordem de palavras de forma randômica.

A hipótese para esta distorção parte do principio que ao remover e trocar de posição um subconjunto pequeno de palavras não corrompe o entendimento do produto. As funções que implementam essa funcionalidade estão disponíveis no Apêndice B.

Figura 9 – Visualização da Distorção de Texto



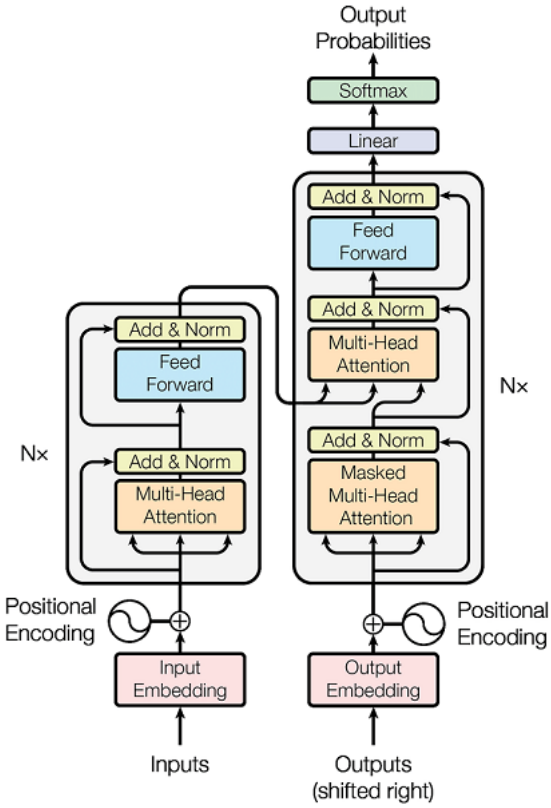
Fonte: Elaborado pelo Autor

2.1.4 ARQUITETURA TRANSFORMER

É utilizado a arquitetura **"Transformer"** proposta no artigo (VASWANI et al., 2017) que é responsável pela a extração dos atributos dos textos.

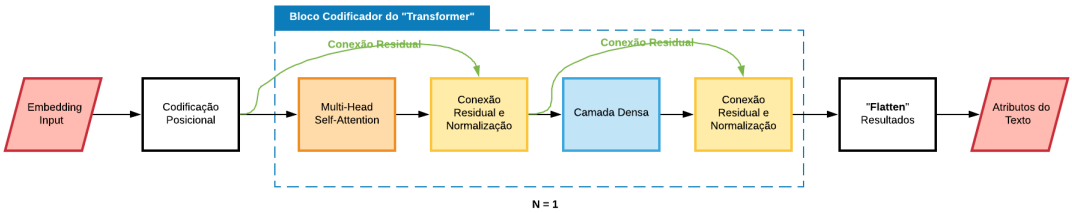
A arquitetura é constituída de dois principais blocos. O Codificador (Encoder) e o Decodificador (Decoder) para fazer a tradução de textos. Dentro do nosso contexto, que envolve a categorização de produtos, utilizamos apenas o bloco **Codificador**. A ilustração desse bloco pode ser visto na Figura 11.

Figura 10 – Arquitetura Transformer



Fonte: Disponível em (VASWANI et al., 2017)

Figura 11 – Bloco Codificador da Arquitetura Transformer



Fonte: Elaborado pelo Autor

2.1.4.1 EMBEDDING INPUT

O "Embedding Input" é um $X_{b,s,d}$ é a representação do texto em formato vetorial. Esses vetores são criados à partir do modelo de FastText introduzido por (BOJANOWSKI et al., 2016) e distorcido. A distorção pode ser entendida no subseção 2.1.3.

O tamanho da matriz é dados por b é o tamanho do lote (batch), s é o tamanho da sequência (quantidade de palavras) e d é a quantidade de dimensões do vetor das palavras.

O tamanho de s varia conforme o conjunto de dados de treinamento, e busca informar qual o tamanho adequado da quantidade máxima de palavras que será utilizado para formar as sequências removendo as discrepâncias ("outliers") e reduzindo assim a complexidade do modelo. Essa quantidade de palavras é dada pela função:

$$A = \{c_1, ..., c_n\}$$
$$k = \lceil p(n+1)/100 \rceil$$

Onde A é uma sequência ordenada da quantidade de palavras por amostra do conjunto de dados de treinamento, p-ésimo é a parte do conjunto de dados que estamos buscando (foi utilizado o valor de p igual à 99,7) e n é o tamanho do conjunto de dados, ambos os valores são calculados à partir do conjunto de dados de treinamento. O resultado k é o posição de A que representa a quantidade de palavras que será utilizada.

2.1.4.2 CODIFICAÇÃO POSICIONAL

Nesta etapa, o objetivo é adicionar a informação de posição nas sequênclatex colchetesias. Isso se faz necessário pois o modelo não possui consciência do que é posição, diferente de uma **Rede Neural Recorrente**. A função para a codificação das sentenças se dá por:

$$PE(pos, i) = \begin{cases} sin(pos/10000^{2i/d}), & i \in [0, i//2) \\ cos(pos/10000^{2i/d}), & i \in [i//2, i] \end{cases}$$

Onde d é o tamanho do vetor dos palavras e i é um vetor de tamanho d que representa a posição de cada atributo do vetor de palavra.

$$i = \Big[0, 1, ..., d-1\Big]$$

Enquanto $pos_{b,s}$ é uma matriz onde b é o tamanho do lote e s é a quantidade de palavras na sequência.

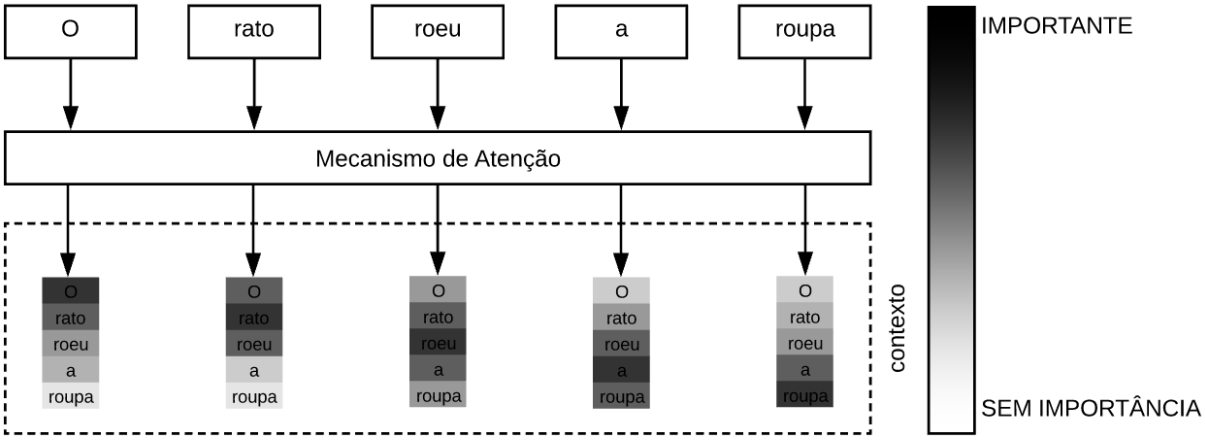
$$pos_{b,s} = \begin{bmatrix} 0 & 1 & 2 & ... & s-1 \\ 0 & 1 & 2 & ... & s-1 \\ ... \\ 0 & 1 & 2 & ... & s-1 \end{bmatrix}$$

A implementação do **Codificação Posicional** pode ser visto no Apêndice E.

2.1.4.3 ATTENTION MECHANISM

O **Attention Mechanism** (Mecanismo de Atenção em tradução direta) introduzido por (BAHDANAU; CHO; BENGIO, 2014) é um dois conceitos mais interessantes no momento. A ideia consiste em **"prestar atenção"** em uma parte da informação em detrimento de tudo. Uma visualização desse mecanismo pode ser vista na Figura 12.

Figura 12 – Exemplo de utilização do mecanismo de atenção em uma sequência



Fonte: Elaborado pelo Autor

Dado uma sequencia de palavras h_x $(h_1, .., h_T)$ é necessário computar um contexto c_i para cada h_i. Criando assim um contexto individual por palavras que pode ser dado pela função de soma ponderada como pode ser visto na função à baixo.

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

Os pesos α_{ij} da soma ponderada é dado pela função **softmax**

$$\alpha_{ij} = \frac{exp(e_{ij})}{\sum_{k=1}^{T_x} exp(e_{ik})}$$

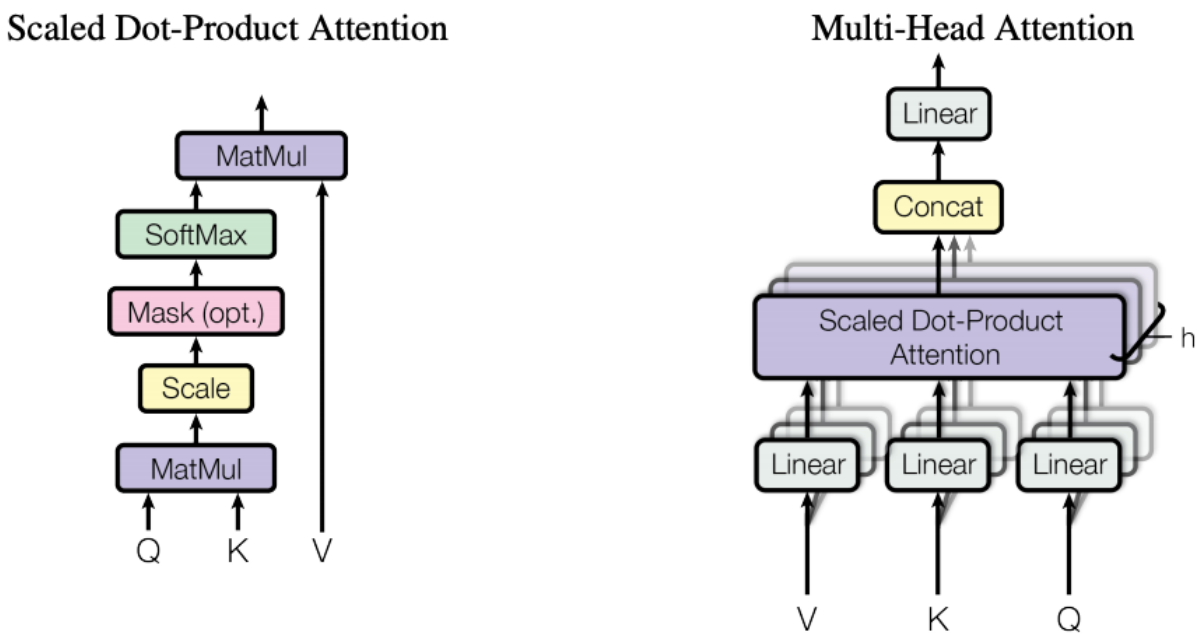
onde e_{ij} é a função de alinhamento do modelo que diz o quão importante é h_j é para a saída s_{i-1}.

$$e_{ij} = a(s_i - 1, h_j)$$

2.1.4.4 MULTI-HEAD ATTENTION

Muli-Head Attention (Atenção de Múltiplas Cabeças em tradução direta) introduzido por (VASWANI et al., 2017) consiste em utilizar o **Attention Mechanism** proposto no artigo (BAHDANAU; CHO; BENGIO, 2014) em projeções à partir da sequencia de entrada. Essas projeções são feitas re-organizando o os vetores das palavras d_{model} em h projeções.

Figura 13 – Scaled Dot-Product Attention e Muilt-Head Attention



Fonte: (VASWANI et al., 2017)

Scalled Dot-Product Attention Mechanism

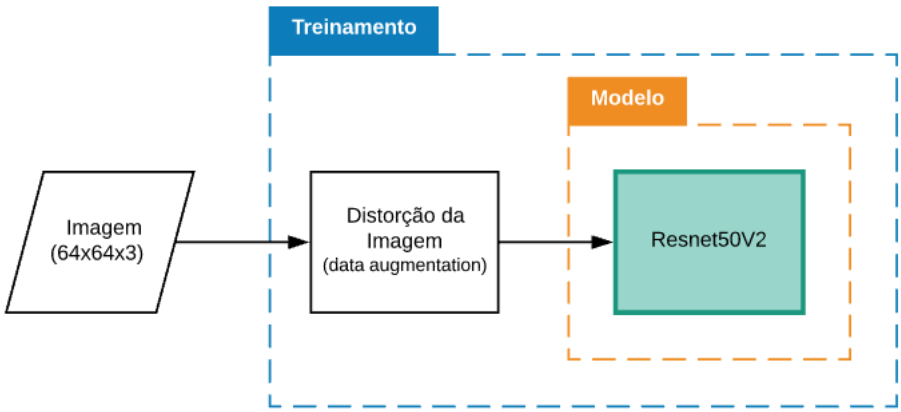
Na arquitetura Transformer, é implementado uma variação do **Attention Mechanism** introduzido por (BAHDANAU; CHO; BENGIO, 2014) que é a escala do produto entre a **Query** e a **Key**.

$$Attention(Q, K, V) = \frac{Q * K}{sqrt(d_{model})} * V$$

2.2 IMAGEM

Para a extração dos atributos das imagens é utilizado o modelo da ResNet proposto no artigo (HE et al., 2015) além das imagens passarem por distorções em cada nova interação. O formato da estada das imagens é no formato **RGB 64x64x3**.

Figura 14 – Arquitetura do modelo de Imagem



Fonte: Elaborado pelo Autor

2.2.1 DISTORÇÃO DAS IMAGENS

Assim como foi feito nos textos, também é feito a distorção (data augmentation) nas imagens para tornar o treinamento melhor e mais genérico, pois toda vez que uma imagem passa pela rede, é como se fosse uma nova imagem. Para atingir esse objetivo é empregado diversas modificações nas imagens como pode ser visto no exemplo na Figura 15 e no Apêndice C pode ser encontrado a implementação.

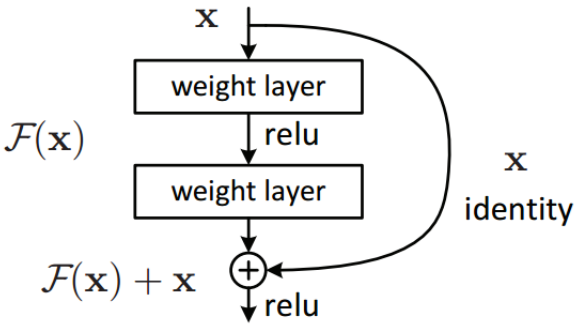
Figura 15 – Distorção de Imagens

Fonte: Elaborado pelo Autor

2.2.2 ARQUITETURA RESNET

Para a extração das características das imagens é utilizado o modelo **ResNet** introduzido no (HE et al., 2015), onde propõe a utilização do **bloco residual** de tal forma que previna o **vanishing e exploding** gradientes em redes neurais muito profundas. O bloco residual pode ser visto na Figura 16.

Figura 16 – Bloco Residual



Fonte: (HE et al., 2015)

Vale notar que o bloco residual não aumenta a complexidade da rede, pois a sua função é combinar o bloco interno com a própria identidade, também chamado de **"shortcut connections"**.

$$F(x) + x$$

Para este trabalho, utilizamos como base a **ResNet** com cinquenta (50) camadas pré-treinadas. Sabemos que existem muitas outras arquiteturas melhores, no entanto, todas trazem consigo maior complexidade, traduzindo para a realidade, um maior tempo de treinamento. Com essa versão das ResNet conseguimos obter excelentes resultados.

Figura 17 – Arquiteturas dos blocos convolucionais da ResNet

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Fonte: (HE et al., 2015)

2.3 CONJUNTO DE DADOS

Como não há uma possibilidade no momento de criar um conjunto de dados anotados manualmente, utilizamos os produtos já classificados previamente. Com isso, o conjunto de dados é separado em três partes:

- Conjunto de dados de treinamento (inclui técnica de oversampling e undersampling)
- Conjunto de dados de validação (utilizado para acompanhar a performance do modelo durante o treinamento é usado na cláusula de parada)
- Conjunto de dados de teste (utilizado para criar o relatório final)

2.3.1 SELEÇÃO DO CONJUNTO DE DADOS

Para tentar criar um conjunto de dados razoável precisamos aplicar algumas técnicas para limpeza e normalização dos dados:

- Para tentar reduzir o ruído no conjunto de dados, é utilizado um modelo de auxiliar capaz de identificar possíveis produtos mal categorizados;
- Quanto os produtos de uma subfamília possuem quantidade menor ou igual à 3, essas subfamílias são removidas pois não possuem dados o suficiente para treinar, validar e testar;

- Uma vez o conjunto de dados selecionado, é feito a separação dos dados de treinamento, validação e teste. Onde 90% dos dados são para treinamento, 5% para validação e 5% para teste. Os dados de treinamento passam ainda por **oversampling** e **undersampling**;

  Undersampling

  Para as subfamílias com muitos produtos, é feito o **undersampling** definindo por

$$mean + 2 * std$$

  Oversampling

  Enquanto o oversampling é utilizado para famílias com menos de 30 exemplos. É feito a replicação de alguns desses meses exemplos para ter um total de 30 exemplos no final.

2.4 OTIMIZAÇÃO E FUNÇÃO DE ERRO

Para a otimização e erro é utilizado o SGD e o Focal Loss que serão descritos à seguir.

2.4.1 OTIMIZADOR

O otimizador utilizado é o **Stochastic Gradient Descent** (SGD) com **Momentum** e **Nesterov** onde obtemos melhores resultados. Usando um **Learning Rate** inicial de 0,01 com um **Learning Rate Scheduler Exponencial** sendo acionado após a 10ª época.

2.4.2 FUNÇÃO DE ERRO (LOSS FUNCTION)

Devido ao desbalanceado dados nas classes, problema que se agrava conforme a especificidade da categoria aumenta, buscamos na literatura formas para mitigar este problema, além da utilização de oversampling e undersampling passamos à utilizar também uma função de erro para ajudar neste processo.

No artigo **Focal Loss for Dense Object Detection** introduzido por (LIN et al., 2017) implementa uma variação do **Cross Entropy** focada em classes extremamente desbalanceadas. A implementação pode ser visto no Apêndice F.

$$FL(p) = -\alpha((1-p)^l) * log(p)$$

3 Resultados

Os resultados desse trabalho, foram todos feitos com dados reais e representação a qualidade do modelo neste momento.

3.1 MÉTODOS DE AVALIAÇÃO

A comparação da performance dos modelos foram feitas utilizando a métrica F1-Score Macro a fim de avaliar a qualidade entre os modelos.

3.2 CONFIGURAÇÃO DO TREINAMENTO

Todos os treinamentos ocorreram utilizando 8xV100 com o tamanho do lote de 128 (por GPU) e a cláusula de parada ocorre quando o loss do conjunto de dados de validação para de cair por um prazo de 10 épocas.

3.3 EXPERIMENTOS

Os resultados dos experimentos podem ser visualizados na Tabela 3

Tabela 3 – Resultados dos Experimentos

	Departamento	Setor	Família	Subfamília	Média	Época	Horas
Texto + Imagem					0.9	50	10h
Imagem							
Texto							

Fonte: Elaborado pelo autor

4 Conclusão

bla bla bla

Referências

BAHDANAU, D.; CHO, K.; BENGIO, Y. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2014. Citado 2 vezes nas páginas 19 e 20.

BOJANOWSKI, P. et al. *Enriching Word Vectors with Subword Information*. 2016. Citado 2 vezes nas páginas 13 e 18.

HE, K. et al. *Deep Residual Learning for Image Recognition*. 2015. Citado 3 vezes nas páginas 20, 22 e 23.

LIN, T.-Y. et al. *Focal Loss for Dense Object Detection*. 2017. Citado na página 24.

MIKOLOV, T. et al. *Efficient Estimation of Word Representations in Vector Space*. 2013. Citado 3 vezes nas páginas 13, 14 e 15.

VASWANI, A. et al. *Attention Is All You Need*. 2017. Citado 3 vezes nas páginas 16, 17 e 20.

Apêndices

APÊNDICE A – FUNÇÃO DE NORMALIZAÇÃO DE TEXTO

O código pode ser acessado no repositório do projeto em: <https://github.com/allanbatista/classificacao-de-produtos-no-e-commerce/blob/master/codigo/notebooks/Cleaner.ipynb>.

APÊNDICE B – FUNÇÃO DE DISTORÇÃO DE TEXTO

O código pode ser acessado no repositório do projeto em: <https://github.com/allanbatista/classificacao-de-produtos-no-e-commerce/blob/master/codigo/notebooks/Distor%C3%A7%C3%A3o_de_Texto.ipynb>.

APÊNDICE C – FUNÇÃO DE DISTORÇÃO DE IMAGEM

O código pode ser acessado no repositório do projeto em <https://github.com/allanbatista/classificacao-de-produtos-no-e-commerce/blob/master/codigo/notebooks/Distor%C3%A7%C3%A3o_de_Imagens_com_Tensorflow.ipynb>.

APÊNDICE D – FASTTEXT MODELO PRÉ-TREINADO

D.1 Configurações do Treinamento

Tabela 4 – Configurações de Treinamento do FastText.

epocas	50
dimensão	128
n-grams	1 até 3
n-grams por palavra	2 até 7
tamanho do dicionário	5000000

Fonte: Elaborado pelo Autor

D.2 Modelos Pré-Treinados

Modelo do FasText: <https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/model.bin>

Modelo Embeddins das palavras: <https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/model.vec>

D.3 Corpus

<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000000.txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000001.txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000002.txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000003.txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000004.txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000005.txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000006.

txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000007.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000008.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000009.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000010.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000011.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000012.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000013.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000014.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000015.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000016.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000017.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000018.
txt>
<https://storage.googleapis.com/black-magic-us-west1/fasttext/2020-07-16T00-00-00/000000000019.
txt>

APÊNDICE E – CODIFICAÇÃO POSICIONAL

O código pode ser acessado no repositório do projeto em <https://github.com/allanbatista/classificacao-de-produtos-no-e-commerce/blob/master/codigo/notebooks/Transformer_Codificacao_Posicional.ipynb>.

APÊNDICE F – FOCAL LOSS

O código pode ser acessado no repositório do projeto em <https://github.com/allanbatista/classificacao-de-produtos-no-e-commerce/blob/master/codigo/notebooks/Focal_Loss_for_Dense_Object_Detection.ipynb>.